ACM/IDS 104 APPLIED LINEAR ALGEBRA
PROBLEM SET 3: SOLUTIONS

KONSTANTIN M. ZUEV

Problem 1. (10 points) Inner Products vs Norms
We know that an inner product $\langle \cdot, \cdot \rangle$ on a vector space V induces a norm on V:

$$\|v\| = \sqrt{\langle v, v \rangle}. \tag{1}$$

(a) (5 points) Suppose we know the induced norm in (1), i.e. we can compute $\|v\|$ for any $v \in V$. Is it possible to reconstruct the inner product $\langle \cdot, \cdot \rangle$ from the norm? That is, to find $\langle u, v \rangle$ for any $u, v \in V$.
(b) (5 points) Are there two distinct inner products that induce the same norm? In other words, can you find $\langle \cdot, \cdot \rangle_1$ and $\langle \cdot, \cdot \rangle_2$, $\langle \cdot, \cdot \rangle_1 \neq \langle \cdot, \cdot \rangle_2$, such that $\| \cdot \|_1 = \| \cdot \|_2$?

Solution:

(a) Intuitively it may seem that it is impossible. But it turns out that *if* a norm is induced by an inner product, then the we can reconstruct the former from the later. Indeed,

$$\|u + v\|^2 = \langle u + v, u + v \rangle = \|u\|^2 + \|v\|^2 + 2\langle u, v \rangle. \tag{2}$$

And, thus,

$$\langle u, v \rangle = \frac{1}{2}(\|u + v\|^2 - \|u\|^2 - \|v\|^2). \tag{3}$$

An alternative, but equivalent expression is

$$\langle u, v \rangle = \frac{1}{4}(\|u + v\|^2 - \|u - v\|^2). \tag{4}$$

The expressions (3) and (4) are called the *polarization identities*.

(b) Suppose $\langle \cdot, \cdot \rangle_1$ and $\langle \cdot, \cdot \rangle_2$ are two distinct inner product. Then there exist vectors $u, v \in V$ such that $\langle u, v \rangle_1 \neq \langle u, v \rangle_2$. On the other hand, since $\langle \cdot, \cdot \rangle_1$ and $\langle \cdot, \cdot \rangle_2$ induce the same norm, $\|u + v\|_1 = \|u + v\|_2$. But this leads to the following:

$$\langle u + v, u + v \rangle_1 = \langle u + v, u + v \rangle_2 \tag{5}$$

$$\|u\|_1^2 + \|v\|_1^2 + 2\langle u, v \rangle_1 = \|u\|_2^2 + \|v\|_2^2 + 2\langle u, v \rangle_2 \tag{6}$$

$$\langle u, v \rangle_1 = \langle u, v \rangle_2, \tag{7}$$

which contradicts our assumption. Thus, distinct inner products induce distinct norms.

Problem 2. (10 points) Inner Products on Function Spaces
Let $C^1[0, 1]$ be a vector space of continuously differentiable functions on the interval $[0, 1]$. Let

$$\langle f, g \rangle_1 = \int_0^1 f'(x)g'(x)dx \quad \text{and} \quad \langle f, g \rangle_2 = \int_0^1 (f(x)g(x) + f'(x)g'(x))dx. \tag{8}$$

(a) (2 points) One of the two products in (8) defines an inner product on $C^1[0, 1]$. Determine which one and explain why.
(b) (2 points) Denote the inner product from part (a) by $\langle \cdot, \cdot \rangle$. Write down explicitly the Cauchy–Schwarz and triangle inequalities based on $\langle \cdot, \cdot \rangle$.
(c) (2 points) Find the angle between $f(x) = 1$ and $g(x) = e^x$ with respect to $\langle \cdot, \cdot \rangle$.

E-mail address: kostia@caltech.edu.

Remark: The norm induced by $\langle \cdot, \cdot \rangle$ belongs to the family of Sobolev norms. Sobolev spaces are natural homes for solutions of PDEs (more natural that spaces of continuously differentiable functions).

Let $C[a, b]$ be a vector space of continuous functions on the interval $[a, b]$, and let $f \in C[a, b]$.

(d) (4 points) Prove that the square of the average of a function is never larger than the average of its square:

$$\left(\frac{1}{b-a} \int_a^b f(x)dx \right)^2 \leq \frac{1}{b-a} \int_a^b f(x)^2 dx. \tag{9}$$

Solution:

(a) $\|f\|_1^2 = \langle f, f \rangle_1 = \int_0^1 f'(x)^2 dx = 0$ does not necessarily mean $f = 0$, it just means $f' = 0$. So $\langle \cdot, \cdot \rangle_1$ is not an inner product. It can be checked that $\langle \cdot, \cdot \rangle_2$ is an inner product (but the problem does not ask to do that).

(b) Cauchy-Schwarz inequality:

$$\begin{aligned} &\left(\int_0^1 (f(x)g(x) + f'(x)g'(x))dx \right)^2 \leq \\ &\int_0^1 (f(x)^2 + f'(x)^2)dx \int_0^1 (g(x)^2 + g'(x)^2)dx \end{aligned} \tag{10}$$

Triangle inequality:

$$\begin{aligned} &\sqrt{\int_0^1 ((f(x) + g(x))^2 + (f'(x) + g'(x))^2)dx} \leq \\ &\sqrt{\int_0^1 (f(x)^2 + f'(x)^2)dx} + \sqrt{\int_0^1 (g(x)^2 + g'(x)^2)dx} \end{aligned} \tag{11}$$

(c) The angle, θ between $f(x) = 1$ and $g(x) = e^x$ can be calculated using the formula $\cos\theta = \frac{\langle f,g \rangle}{\|f\|\|g\|}$

$$\begin{aligned} \langle f, g \rangle &= \int_0^1 e^x dx = e - 1 \\ \|f\|^2 &= \int_0^1 1dx = 1 \\ \|g\|^2 &= \int_0^1 e^{2x} dx + \int_0^1 e^{2x} dx = e^2 - 1 \end{aligned} \tag{12}$$

Thus, $\cos\theta = \frac{e-1}{\sqrt{e^2-1}} = \sqrt{\frac{e-1}{e+1}} \approx 0.67$, and $\theta = 0.82$ radians.

(d) This analysis problem admits an elegant linear algebraic solution. By looking at the inequality (9), we see that it seems to have the form of the Cauchy–Schwarz inequality,

$$|\langle u, v \rangle| \leq \|u\|\|v\|. \tag{13}$$

We just need to find the appropriate inner product $\langle \cdot, \cdot \rangle$ and functions u and v such that (13) yields (9). Let's consider the L^2 inner product on $[a, b]$,

$$\langle u, v \rangle = \int_a^b u(x)v(x)dx, \tag{14}$$

and apply it to $u = f$ and $v = 1$:

$$\left| \int_a^b f(x) \cdot 1dx \right| \leq \sqrt{\int_a^b f(x)^2 dx} \sqrt{\int_a^b 1^2 dx} = \sqrt{b-a} \sqrt{\int_a^b f(x)^2 dx}. \tag{15}$$

Squaring both sides and dividing by $(b-a)^2$ gives

$$\left(\frac{1}{b-a}\int_a^b f(x)dx\right)^2 \leq \frac{1}{b-a}\int_a^b f(x)^2 dx. \tag{16}$$

Problem 3. (10 POINTS) THE K-MEANS ALGORITHM FOR CLUSTERING
Complete Problem 3 in `PS3.mlx`.

Solution: See `PS3solutions.mlx` (uploaded to the Piazza page). The required plots are shown below. For this data set, as expected, our implementation and the the built-in one give identical results.

Problem 4. (10 POINTS) GRAM MATRICES
Consider a vector space $C^1[0,1]$ of continuously differentiable functions on $[0,1]$ with the L^2 inner product.

(a) (3 points) Find the Gram matrix G associated with $1, e^x, e^{2x}$.
(b) (3 points) Is G positive definite?
(c) (3 points) Answer (a) and (b) using the inner product from Problem 2.
(d) (1 point) Let $\langle\cdot,\cdot\rangle_1$ and $\langle\cdot,\cdot\rangle_2$ be two different inner products on $C^1[0,1]$, and G_1 and G_2 be the Gram matrices associate with $1, e^x, e^{2x}$ computed with respect to $\langle\cdot,\cdot\rangle_1$ and $\langle\cdot,\cdot\rangle_2$. Can you find $\langle\cdot,\cdot\rangle_1$ and $\langle\cdot,\cdot\rangle_2$ such that one of the Gram matrices is positive definite and the other one is not?

Solution:

(a) Let's compute the inner products that form the entries of the Gram matrix:

$$\begin{aligned}
\langle 1,1\rangle &= \int_0^1 1dx = 1,\\
\langle 1,e^x\rangle &= \langle e^x,1\rangle = \int_0^1 e^x dx = e-1,\\
\langle 1,e^{2x}\rangle &= \langle e^x,e^x\rangle = \langle e^{2x},1\rangle = \int_0^1 e^{2x}dx = \frac{1}{2}(e^2-1),\\
\langle e^x,e^{2x}\rangle &= \langle e^{2x},e^x\rangle = \int_0^1 e^{3x}dx = \frac{1}{3}(e^3-1)\\
\langle e^{2x},e^{2x}\rangle &= \int_0^1 e^{4x}dx = \frac{1}{4}(e^4-1).
\end{aligned} \tag{17}$$

This leads to the following Gram matrix:

$$G = \begin{bmatrix} 1 & e-1 & \frac{1}{2}(e^2-1)\\ e-1 & \frac{1}{2}(e^2-1) & \frac{1}{3}(e^3-1)\\ \frac{1}{2}(e^2-1) & \frac{1}{3}(e^3-1) & \frac{1}{4}(e^4-1)\end{bmatrix} \tag{18}$$

(b) Since G is a Gram matrix, it is positive semi-definite. It is positive definite if and only if the vectors G associated with are linearly independent. Let's use the results of Practice Problem 3B to show that functions $1, e^x, e^{2x}$ are linearly independent, and, thus, G is positive definite. The Wronskian matrix of $f_1 = 1, f_2 = e^x, f_3 = e^{2x}$ is

$$W(x) = \begin{bmatrix} f_1(x) & f_2(x) & f_3(x)\\ f_1'(x) & f_2'(x) & f_3'(x)\\ f_1''(x) & f_2''(x) & f_3''(x)\end{bmatrix} = \begin{bmatrix} 1 & e^x & e^{2x}\\ 0 & e^x & 2e^{2x}\\ 0 & e^x & 4e^{2x}\end{bmatrix} \tag{19}$$

The Wronskian at $x=0$ is

$$\det W(0) = \det\begin{bmatrix} 1 & 1 & 1\\ 0 & 1 & 2\\ 0 & 1 & 4\end{bmatrix} = 2 \neq 0. \tag{20}$$

Therefore, the functions are linearly independent (for details, see Practice Problem 3B), and G is positive definite.

(c) Let's compute the inner products that form the entries of the Gram matrix:

$$\begin{aligned}
\langle 1, 1\rangle &= \int_0^1 1dx = 1, \\
\langle 1, e^x\rangle = \langle e^x, 1\rangle &= \int_0^1 e^x dx = e - 1, \\
\langle 1, e^{2x}\rangle = \langle e^{2x}, 1\rangle &= \int_0^1 e^{2x} dx = \frac{1}{2}(e^2 - 1), \\
\langle e^x, e^x\rangle &= \int_0^1 e^{2x} + e^{2x} dx = e^2 - 1, \\
\langle e^x, e^{2x}\rangle = \langle e^{2x}, e^x\rangle &= \int_0^1 e^{3x} + 2e^{3x} dx = (e^3 - 1), \\
\langle e^{2x}, e^{2x}\rangle &= \int_0^1 e^{4x} + 4e^{4x} dx = \frac{5}{4}(e^4 - 1).
\end{aligned} \tag{21}$$

This leads to the following Gram matrix:

$$H = \begin{bmatrix} 1 & e-1 & \frac{1}{2}(e^2-1) \\ e-1 & e^2-1 & e^3-1 \\ \frac{1}{2}(e^2-1) & e^3-1 & \frac{5}{4}(e^4-1) \end{bmatrix} \tag{22}$$

It is positive definite by exactly the same reason as above.

(d) No, we can't. Since 1, e^x, e^{2x} are linearly independent, the Gram matrix associated with these vectors is always positive definite, regardless of chosen inner product.

Problem 5.[1] (10 points) Gram Matrices for Text Classification
Complete Problem 5 in `PS3.mlx`.

Solution: See `PS3solutions.mlx` (uploaded to the Piazza page).

Acknowledgements. A huge thanks to Pavlos Stavrinides for editing my clumsy MATLAB m-files and converting them into neat live scripts!

[1]Inspired by New York's presidential primary, April 19, 2016.

ACM/IDS 104 - Problem Set 3 - MATLAB Solutions

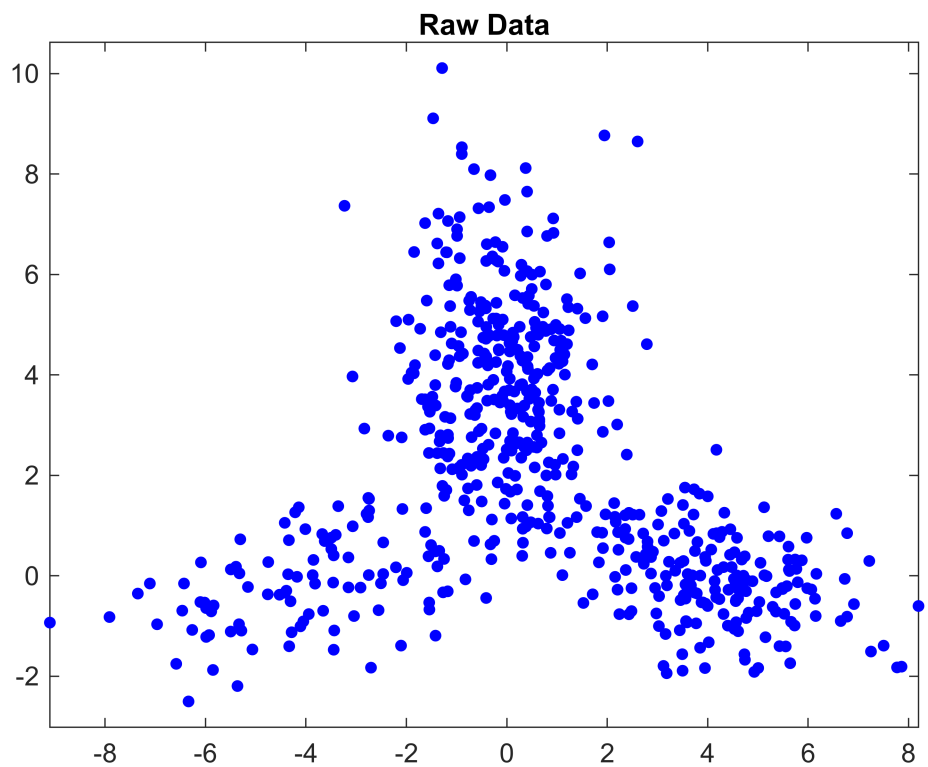
Before writing your MATLAB code, it is always good practice to get rid of any leftover variables and figures from previous scripts.

```
clc; clear; close all;
```

Problem 3 (10 points) The K-Means Algorithm for Clustering

In this problem, we will be implementing the K-Means Algorithm for clustering vectors in $\mathbb{R}^2$. To start, let us load and visualize our dataset.

```
load clustering_data.mat
figure;
plot(x(:,1),x(:,2),'.b','MarkerSize',15);
title("Raw Data")
axis equal;
```




```
rng(2016); % for reproducibility
```

Part (a) The Algorithm

In this part, we will implement the algorithm for $K = 3$, which seems to be a reasonable number of clusters for this dataset. Show the clusters after the 1st, 5th, 10th and last iteration. This will illustrate the cluster evolution as the algorithm runs. `subplot()` is very useful here.

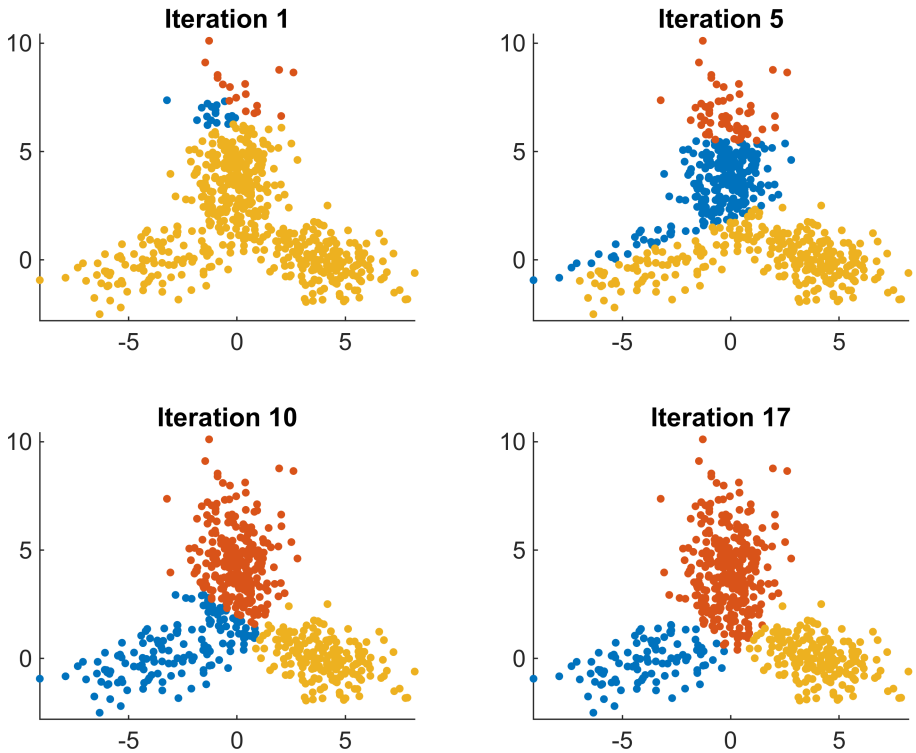
To further aid our understanding of the algorithm, we will be keeping track of a few things:

1. Keep track of the evolution of the representatives for each cluster. You can do so by updating the variable `z` after each iteration.
2. Keep track of the objective function value at each iteration.

Once you are done implementing the algorithm, feel free to change the value of `K` and see how the clusterings are altered.

```
%{
Some useful setup done for you
%}
K = 3; % number of clusters
n = length(x); % number of vectors
c = zeros(n, 1); % clustering : c(i) is the cluster number for vector x(i, :)
iter = 0; % iteration number
z = datasample(x, K); % initial representatives

%{
Step 1: Partitioning into clusters given our initial representatives
%}
for i = 1 : n
    for k = 1 : K
        d(k) = norm(z(k, :) - x(i, :));  % distance to the representative
    end
    [rho(i),ind] = min(d); % find the minimum distance
    c(i, 2) = ind; % update cluster index
end
%{
Increment the iteration number
Update the objective function
%}
iter = iter + 1; % increment iteration number
Rho(iter) = mean(rho.^2); % objective function

%{
Iterate until convergence:
%}
while norm(c(:, iter + 1) - c(:, iter)) ~= 0
    iter = iter + 1;  % new iteration
    % Step 2: given clustering, find representatives
    for k = 1 : K
        z(k, :, iter) = mean(x(c(:, iter) == k, :));
    end
    % Step 1: given representatives, partition vectors into K clusters
    for i = 1 : n
        for k = 1 : K
            d(k) = norm(z(k, :, iter) - x(i, :));  % distance to the representative
        end
```

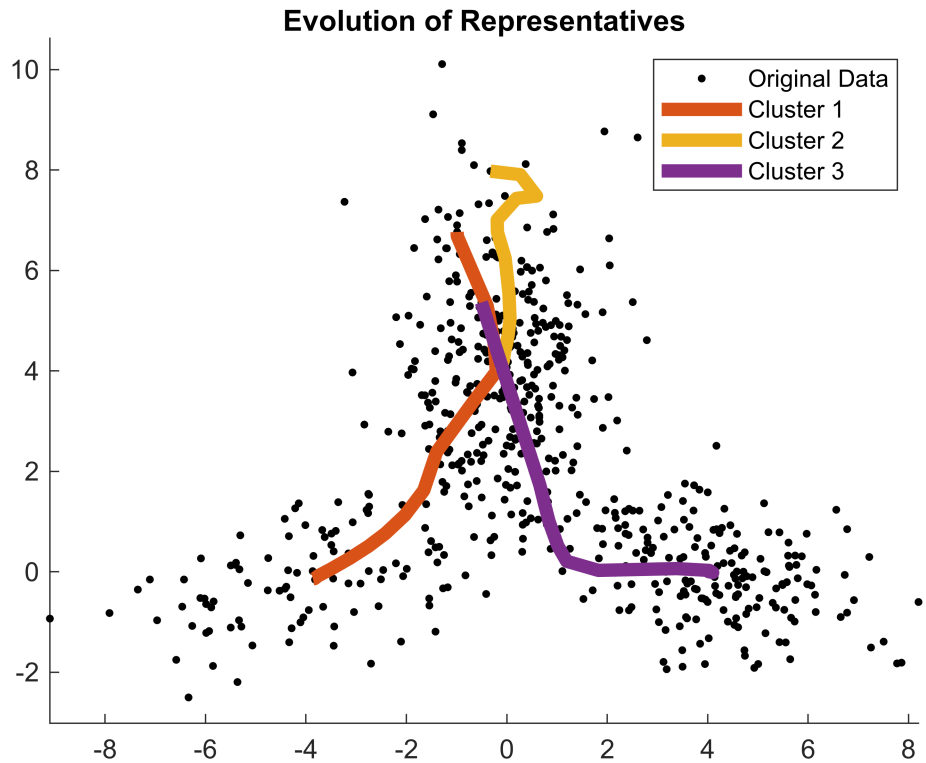
```
        [rho(i),ind] = min(d); % find the minimum distance
        c(i, iter + 1) = ind; % update cluster index
    end
    Rho(iter) = mean(rho.^2); % objective function
end

%{
Plotting
%}
figure
subplot(2,2,1)  % Iteration 1
hold on
for k=1:K
    plot(x(c(:,2)==k,1),x(c(:,2)==k,2),'.','MarkerSize',10)
end
title("Iteration 1");
axis equal
subplot(2,2,2) % Iteration 5
hold on
for k=1:K
    plot(x(c(:,6)==k,1),x(c(:,6)==k,2),'.','MarkerSize',10)
end
title("Iteration 5");
axis equal
subplot(2,2,3) % Iteration 10
hold on
for k=1:K
    plot(x(c(:,11)==k,1),x(c(:,11)==k,2),'.','MarkerSize',10)
end
title("Iteration 10");
axis equal
subplot(2,2,4)  % Final clusering
hold on
for k=1:K
    plot(x(c(:,iter+1)==k,1),x(c(:,iter+1)==k,2),'.','MarkerSize',10) %
end
title(strcat("Iteration ", int2str(iter)));
axis equal
```



Part (b) Evolution of Representatives

Now, let us see the dynamics of the representatives. Plot the original (raw) data and the trajectories of the cluster representatives in one figure. `squeeze()` might be useful here.

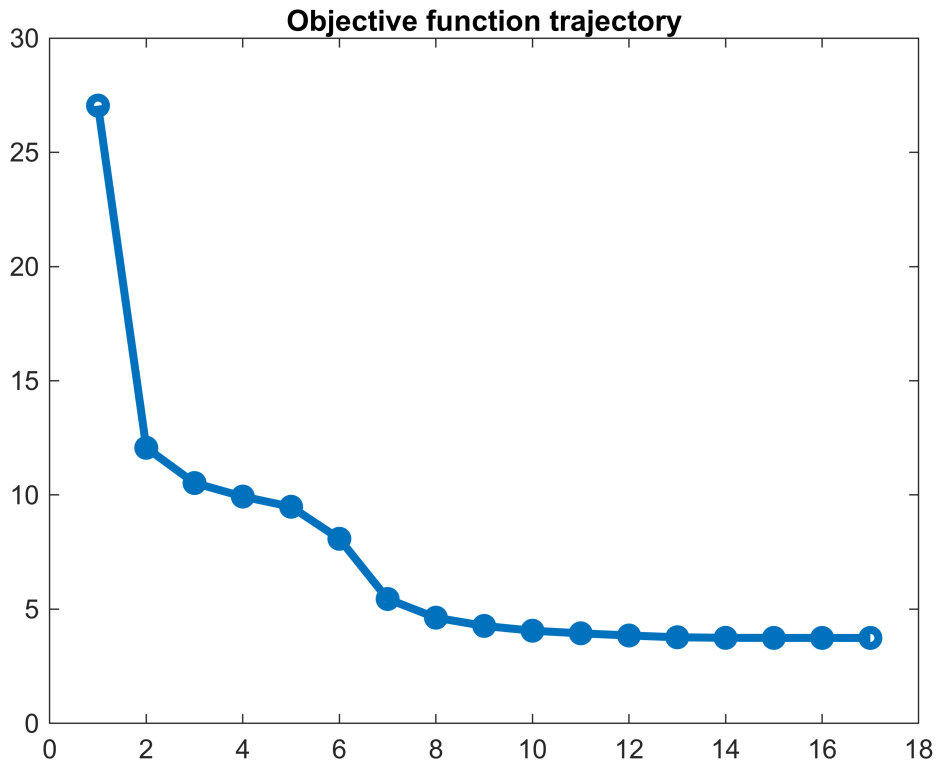
```
figure
hold on
plot(x(:,1),x(:,2),'.k','MarkerSize',10)
plot(squeeze(z(1,1,:)),squeeze(z(1,2,:)),squeeze(z(2,1,:)),squeeze(z(2,2,:)),...
    squeeze(z(3,1,:)),squeeze(z(3,2,:)), "LineWidth", 5)
title("Evolution of Representatives");
legend("Original Data", "Cluster 1", "Cluster 2", "Cluster 3")
axis equal
```



Part (c) Objective Function

Plot the values of the objective function against the iteration number. Does the trend match what we expect? Report the minimum value of the objective function using `disp()`.

```
figure;
plot(Rho,'-o', "LineWidth", 3);
title("Objective function trajectory");
```




```
disp(strcat("The minimum value of the Objective function is ", num2str(min(Rho))))
```

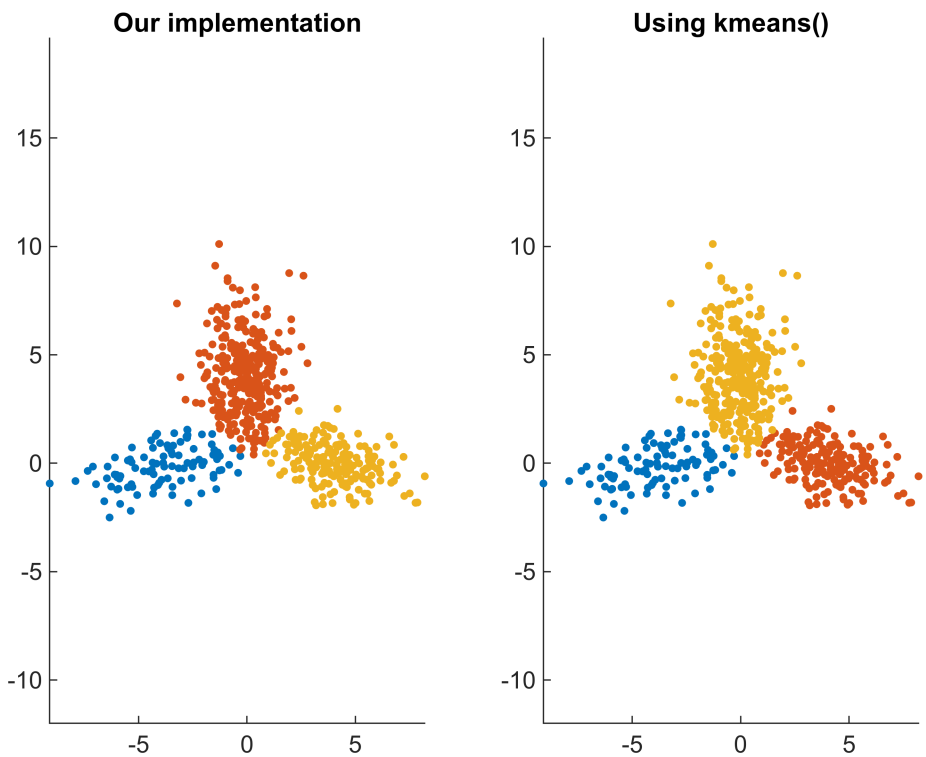
```
The minimum value of the Objective function is 3.742
```

Part (d) Comparing with `kmeans()`

The K-means algorithm is a very popular clustering algorithm and it is often used in various applications. As such, it is implemented in most numerical software packages. In MATLAB, it exists as the built-in function `kmeans()`. In the simplest form, `idx = kmeans(X, k)` performs K-means clustering to partition the rows in the data matrix `X` into `k` clusters, and returns a vector `idx` containing cluster indices for each row. To compare your implementation with the built-in one, plot the clustering your obtained in (a) and the clustering obtained by using `kmeans()`.

```
id = kmeans(x,K);
figure;
subplot(1,2,1)  % Our kmeans
hold on
for k=1:K
    plot(x(c(:,iter+1)==k,1),x(c(:,iter+1)==k,2),'.','MarkerSize',10) %
end
title("Our implementation");
axis equal
subplot(1,2,2)  % Built-in kmeans
hold on
for k=1:K
```

```
    plot(x(id==k,1),x(id==k,2),'.','MarkerSize',10) %
end
title("Using kmeans()");
axis equal
```



Problem 5 (10 points) Gram Matrices for Text Classification

Gram matrices are used in many different applications. Here is a simple example of application to classification of text documents. Suppose we have a collection of n text documents and we want to measure similarity between them. First, we predefine a list of words (the "dictionary") $w_1, w_2, w_3, \cdots, w_m$ appearing in the documents. Then, for each text i we construct a vector $x_i \in \mathbb{R}^m$ whose j^{th} component is the number of times the word w_j appears in the text. This way, each text is represented as a vector in $\mathbb{R}^m$. This representation is often referred to as the "bag of words" representation.

Let $\langle \cdot, \cdot \rangle$ be the usual dot product in $\mathbb{R}^m$ and $\| \cdot \|$ be the Euclidean norm. Let us normalize all vectors $z_i = \frac{x_i}{\|x_i\|}$ and define:

$$Z = [z_1 \cdots z_n] \in \mathbb{M}_{m \times n}$$

Let G be the Gram matrix associated with $z_1 \cdots z_n$:

$$G = Z^T Z \in \mathbb{M}_{n \times n}$$

Since all z_i are unit vectors:

$$G_{ij} = z_i^T z_j = \|z_i\| \|z_j\| \cos\theta_{ij} = \cos\theta_{ij}$$

where θ_{ij} is the angle between z_i and z_j, which of course is the same as the angle between x_i and x_j. This is widely use as a *measure of similarity* between documents. Hence, the more similar two texts i and j are, the closer G_{ij} is to 1.

Now, let us apply this method to real texts. The task is to compare the text of the US Constitution (found here) with the Wikipedia pages of Bernie Sanders, Hillary Clinton, Donald Trump, Ted Cruz, and John Kasich, and to find the page which is "closest" to the Constitution.

Part (a) Setup

To start, we will define a list of $m = 10$ words. To find the most "relevant words", copy the text of the Constitution and paste it into a word-cloud generator (such as this one). High frequency words appear large and we can form the dictionary from these words. Note that common sense is needed: words like "amendment" and "section" would appear large, but it makes no sense to include them into the dictionary. Also, it makes sense to include "United States" as a single "word", not two separate words, etc. But you are free to chose your own dictionary. In the code box below, please fill out the dictionary with your chosen words.

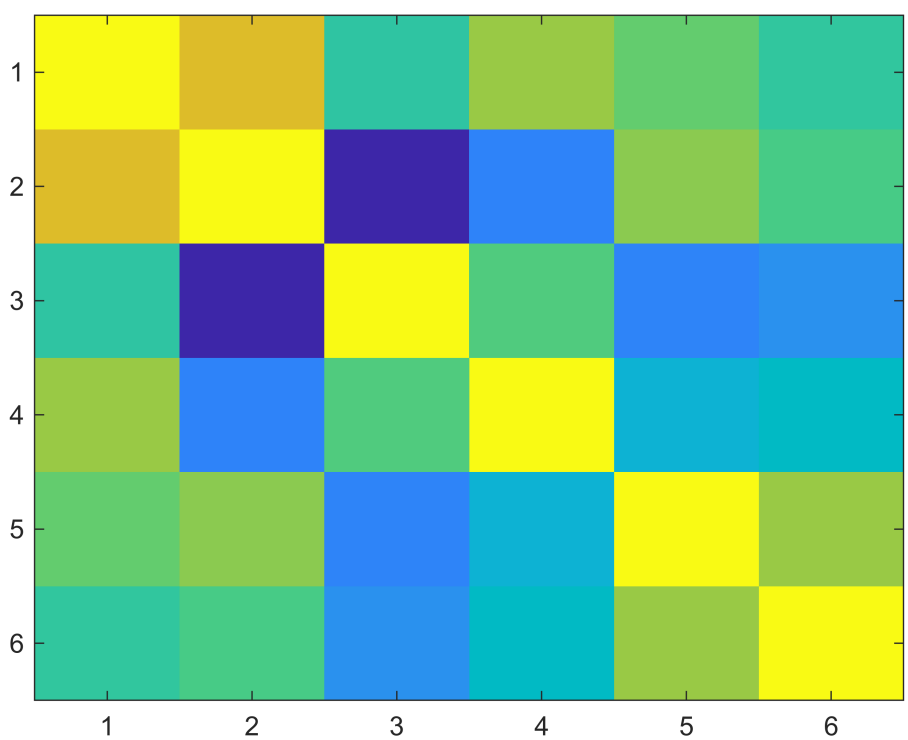
```
%{
DICTIONARY:
1) United States
2) president
3) senate
4) law
5) congress
6) office
7) representative
8) power
9) authority
10) votes
%}
```

Part (b) Fun

With your specified dictionary, let us see which Wikipedia page is closest. First, construct $n = 6$ vectors (as specified above), one for each Wikipedia page and one for the Constitution. No need to get fancy here; just go to the related page and search for the number of occureneces of the words in your dictionary. Put all your results in a column vector for each candidate:

```
%{
Report your results here
%}
Clinton = [49 137 62 45 24 27 6 28 1 17]';
Trump = [56 186 20 51 38 50 6 18 5 15]';
Sanders = [60 103 111 28 60 30 29 14 1 27]';
Cruz = [76 106 83 85 37 22 6 12 1 20]';
```

```
Kasich = [17 70 19 37 27 23 27 9 3 7]';
Constitution = [89 121 29 54 62 57 37 37 8 14]';

%{
Normalize your vectors
%}
Clinton_norm = Clinton / norm(Clinton);
Trump_norm = Trump / norm(Trump);
Sanders_norm = Sanders / norm(Sanders);
Cruz_norm = Cruz / norm(Cruz);
Kasich_norm = Kasich / norm(Kasich);
Const_norm = Constitution / norm(Constitution);

%{
Construct the Gram matrix G as discussed
%}
Z = [Clinton_norm, Trump_norm, Sanders_norm, Cruz_norm, Kasich_norm, Const_norm];
G = Z' * Z;
%{
You can visualize G by uncommenting the following lines of code
%}
figure;
imagesc(G);
```




```
%{
Find the page closest to the Constitution text and report your
answer using disp()
```

```
%}
[~,ind]=max(G(6,1:5));
switch ind
    case 1
        disp('The closest to text "Constitution" is text "Clinton"')
    case 2
        disp('The closest to text "Constitution" is text "Trump"')
    case 3
        disp('The closest to text "Constitution" is text "Sanders"')
    case 4
        disp('The closest to text "Constitution" is text "Cruz"')
    case 5
        disp('The closest to text "Constitution" is text "Kasich"')
end
```

```
The closest to text "Constitution" is text "Kasich"
```

Submit the obtained result to the corresponding piazza poll. Remember: this is not about your political preferences, it is about your data analysis :)

It is interesting to observe that the most dissimilar texts are wiki pages of Trump and Sanders, which makes intuitive sense, since these two politicians have very different views.